Basics

Variables and types

Output & Variables

`println("Hi")`	print line
`val x = 5`	read-only
`var y = 3.14`	changeable
`val s: String = "text"`	explicit type
`const val MAX = 10`	compile-time constant
`readln()`	read input

Types

`Int  Long  Double  Float`	numbers
`String  Char  Boolean`	text / logic
`x.toString()  "5".toInt()`	convert
`"5".toIntOrNull()`	safe convert
`Any  Unit  Nothing`	special types

Operators

`+  -  *  /  %`	math
`==  !=  <  >`	compare (== is equals)
`===`	same object
`&&  ||  !`	logic
`"Hi $name"  "${a + b}"`	string template
`// one line  /* many */`	comments

Strings & Text

Work with text

Common Methods

`s.length  s.isEmpty()`	size / empty?
`s.uppercase()  s.lowercase()`	change case
`s.trim()  s.trimEnd()`	remove spaces
`s.substring(1, 3)`	part of text
`s.split(",")`	text to list
`s.replace("a", "b")`	replace text
`s.contains("a")  "a" in s`	contains?

Check & Search

`s.startsWith("a")  s.endsWith("z")`	start / end
`s.indexOf("a")`	position or -1
`s.first()  s.last()`	first / last char
`s.isBlank()`	only spaces?
`s.isNullOrEmpty()`	null or empty?
`s.take(3)  s.drop(3)`	first / skip chars

Build & Convert

`"""raw text"""`	multi-line raw string
`s.padStart(5, '0')`	pad on the left
`s.reversed()`	reverse text
`s.toList()  s.toCharArray()`	characters
`list.joinToString(", ")`	list to text
`buildString { append("a") }`	string builder

Collections

Lists, maps, sets

List

`val l = listOf(1, 2, 3)`	read-only list
`val m = mutableListOf(1, 2)`	changeable list
`m.add(3)  m.remove(3)`	add / remove
`l.size  l[0]  l.first()`	size / get
`l.contains(2)  2 in l`	has item?

Map & Set

`val d = mapOf("a" to 1)`	read-only map
`val md = mutableMapOf<String, Int>()`	changeable map
`md["a"] = 1`	set value
`d["a"]`	get value (nullable)
`d.getOrDefault("a", 0)`	safe get
`setOf(1, 2, 3)`	unique items

Functional

`l.map { it * 2 }`	transform
`l.filter { it > 1 }`	keep matches
`l.sortedBy { it }  l.sorted()`	sort
`l.sum()  l.max()  l.average()`	totals
`l.groupBy { it % 2 }`	group items
`l.any { }  l.all { }`	any / all

Collection Operations

Transform lists

Transform

`l.map { it * 2 }`	change each item
`l.filter { it > 1 }`	keep matches
`l.flatMap { listOf(it, it) }`	map and flatten
`l.mapIndexed { i, x -> i * x }`	map with index
`l.associateWith { it * 2 }`	items to map
`l.groupBy { it % 2 }`	group into map

Aggregate

`l.sum()  l.average()`	total / mean
`l.max()  l.min()`	largest / smallest
`l.count { it > 1 }`	count matches
`l.fold(0) { a, b -> a + b }`	fold with start
`l.reduce { a, b -> a + b }`	fold to one value
`l.sumOf { it.price }`	sum a property

Order & Pick

`l.sorted()  l.sortedDescending()`	new sorted list
`l.sortedBy { it.name }`	sort by field
`l.take(3)  l.drop(3)`	first / skip items
`l.distinct()`	remove duplicates
`l.chunked(2)  l.windowed(2)`	groups / windows
`l.zip(other)`	pair up items

Control Flow

Decide and repeat

Conditions

`if (x > 0) a else b`	if is an expression
`when (x) { 1 -> "a"; else -> "b" }`	powerful switch
`in 1..5 ->`	range check
`is String ->`	type check
`x in 1..10`	in range?

Loops

`for (i in 0 until 5)`	0 to 4
`for (i in 1..5)`	1 to 5
`for (i in 10 downTo 1 step 2)`	count down
`for (x in list)`	loop items
`for ((i, x) in list.withIndex())`	index + item
`while (cond) { }`	loop while true
`break  continue`	exit / skip

Ranges

`1..10  1 until 10`	inclusive / exclusive
`'a'..'z'`	char range

Operators & Idioms

Write idiomatic code

Operator Overloading

`operator fun plus(o: A): A`	overload +
`operator fun get(i: Int)`	obj[i]
`operator fun invoke()`	obj()
`operator fun compareTo(o: A): Int`	< and >
`infix fun A.to(b: B)`	call without dot

Destructuring

`val (a, b) = pair`	unpack pair
`val (x, y) = point`	unpack data class
`for ((k, v) in map) { }`	loop map entries
`val (_, second) = pair`	ignore a value

Handy Idioms

`if (x in 1..10) { }`	range check
`val v = if (a) 1 else 2`	if as value
`val r = when { a -> 1; else -> 2 }`	when as value
`x ?: return`	exit if null
`TODO("later")`	not implemented yet
`println(x.javaClass)`	runtime class

Functions

Define and use

Functions

`fun add(a: Int, b: Int): Int { }`	define function
`fun add(a: Int, b: Int) = a + b`	single expression
`fun f(a: Int = 1)`	default value
`f(b = 2, a = 1)`	named arguments
`fun f(vararg n: Int)`	many arguments
`fun <T> f(x: T): T`	generic function

Lambdas

`val f = { x: Int -> x * 2 }`	lambda
`list.map { it * 2 }`	implicit it
`fun f(op: (Int) -> Int)`	function parameter
`x.let { }  x.also { }`	scope functions
`x.apply { }  with(x) { }`	configure object

Extensions

`fun String.shout() = uppercase()`	extension function
`inline fun f()`	inline code

Scope Functions

let, run, apply

Pick the Right One

`x.let { it + 1 }`	it, returns result
`x.run { this + 1 }`	this, returns result
`x.apply { y = 1 }`	this, returns x
`x.also { println(it) }`	it, returns x
`with(x) { y }`	this, returns result

Common Uses

`s?.let { use(it) }`	only if not null
`Person().apply { name = "A" }`	configure object
`x.also { log(it) }`	side effect in chain
`run { val t = 1; t + 1 }`	local scope block

Related Helpers

`x.takeIf { it > 0 }`	x or null
`x.takeUnless { it > 0 }`	null or x
`repeat(3) { println(it) }`	run 3 times
`lazy { compute() }`	compute on first use
`use { }`	auto close resource
`check(x > 0)`	throw if false
`require(x > 0)`	validate argument

Classes

OOP in Kotlin

Classes

`class P(val name: String, var age: Int)`	primary constructor
`open class A  class B: A()`	inherit
`override fun f()`	replace parent
`abstract class A`	cannot instantiate
`interface I { fun f() }`	interface
`object Singleton`	single instance
`companion object { }`	static members

Special Classes

`data class U(val id: Int)`	auto equals, copy
`u.copy(id = 2)`	copy and change
`sealed class R`	limited subclasses
`enum class D { UP, DOWN }`	enumeration
`class A { init { } }`	init block
`by lazy { }`	lazy property

Visibility

`public  private  protected  internal`	access levels

Inheritance

Interfaces and abstract

Open & Override

`open class A { }`	allow inheriting
`class B : A() { }`	inherit
`open fun f() { }`	allow override
`override fun f() { }`	replace behavior
`super.f()`	call parent
`final override fun f() { }`	stop further override

Interfaces & Abstract

`interface I { fun f() }`	contract
`interface I { fun g() = 1 }`	default method
`abstract class C { }`	cannot create
`abstract fun f()`	must override
`class D : C(), I { }`	extend + implement

Types & Checks

`x is B`	type check
`x as B`	cast (may fail)
`x as? B`	safe cast
`object Singleton { }`	one instance
`companion object { }`	static-like members
`typealias Id = Int`	type alias

Generics & Delegation

Reuse and share

Generics

`class Box<T>(val v: T)`	generic class
`fun <T> first(l: List<T>): T`	generic function
`<T : Number>`	upper bound
`<out T>  <in T>`	variance
`inline fun <reified T> f()`	keep type at runtime
`Box<*>`	star projection

Delegation

`class A(b: I) : I by b`	delegate interface
`val x by lazy { 5 }`	computed once
`by Delegates.observable(0) { }`	watch changes
`val z by map`	read from a map
`lateinit var s: String`	set later

Properties

`val a get() = x * 2`	computed property
`var b = 0; private set`	private setter
`const val MAX = 10`	compile-time constant
`@JvmStatic`	static for Java

Numbers & Math

Calculations

kotlin.math

`abs(x)  sqrt(x)`	absolute / root
`x.pow(2)`	power
`round(x)  floor(x)  ceil(x)`	rounding
`max(a, b)  min(a, b)`	larger / smaller
`PI  E`	constants
`sin(x)  cos(x)`	trigonometry

Convert & Limits

`x.toInt()  x.toDouble()`	convert type
`Int.MAX_VALUE  Int.MIN_VALUE`	int limits
`1_000_000`	readable number
`0xFF  0b101`	hex / binary
`x.coerceIn(0, 10)`	clamp to range
`x % 2 == 0`	is even?

Random & Format

`Random.nextInt(1, 7)`	number 1 to 6
`Random.nextDouble()`	0.0 to 1.0
`list.random()`	pick one item
`list.shuffled()`	random order
`"%.2f".format(x)`	2 decimals
`x.toString(2)`	binary text

Files & I/O

Read and write

Files

`File("a.txt").readText()`	read whole file
`File("a.txt").readLines()`	lines to list
`File("a.txt").writeText(s)`	write file
`File("a.txt").appendText(s)`	add to file
`File("a.txt").exists()`	file exists?
`File("a.txt").delete()`	delete file

Streams

`File(p).forEachLine { }`	read line by line
`File(p).bufferedReader().use { }`	auto-closing reader
`File(p).walk()`	all files in tree
`File(d).listFiles()`	list folder
`File(d).mkdirs()`	make folders

Console & Time

`readLine() ?: ""`	read a line
`readln()`	read line, non-null
`System.currentTimeMillis()`	time in ms
`measureTimeMillis { }`	time a block
`LocalDate.now()`	today (java.time)
`Thread.sleep(1000)`	pause 1 second

Flow & Channels

Async streams

Flow

`flow { emit(1) }`	create a flow
`flowOf(1, 2, 3)`	flow from values
`list.asFlow()`	list to flow
`f.collect { println(it) }`	receive values
`f.map { it * 2 }.filter { it > 2 }`	operators
`f.toList()`	collect into list

State & Shared

`MutableStateFlow(0)`	observable state
`state.value = 1`	update state
`MutableSharedFlow<Int>()`	broadcast events
`f.stateIn(scope)`	flow to state
`f.catch { }`	handle errors

Channels & Jobs

`Channel<Int>()`	send between coroutines
`ch.send(1)  ch.receive()`	send / receive
`job.cancel()`	stop a coroutine
`withTimeout(1000) { }`	time limit
`coroutineScope { }`	wait for children
`Dispatchers.Default`	CPU work thread

Null Safety & Coroutines

Safe and async

Null Safety

`var s: String? = null`	nullable type
`s?.length`	safe call
`s ?: "default"`	elvis default
`s!!.length`	force (risky)
`s?.let { }`	run if not null
`x as? Int`	safe cast
`lateinit var s: String`	set later

Coroutines

`suspend fun f() { }`	suspendable function
`runBlocking { }`	block until done
`launch { }`	fire and forget
`async { }.await()`	get a result
`delay(1000)`	wait 1 sec
`withContext(Dispatchers.IO) { }`	switch thread
`flow { emit(1) }`	async stream

Good Habits

`Prefer val over var`	safer code
`Avoid !!`	no null crashes

Keep coding. Keep learning.

Made by

@hafizibrahimsuriya

Save this sheet | Share it | Practice daily